

COMET LEAP Master Plan v2.1

Revision: 2.1

Date: 2026-05-26

Prepared for: Unified Streams Convergence

Classification: Internal Operating Plan

Executive Summary

COMET LEAP v2.1 replaces placeholder content with an operationally grounded program plan for converging local-first AI systems, orchestration policy, QA guardrails, and deployment streams across the active repositories. This plan is built from verified workspace artifacts and runtime observations in `C:\tmp\human-flourishing-frameworks-scan` and `C:\Users\alexp\Documents\gm-agent-orchestrator`.

Primary outcome: a reliable, auditable, staff-engineer-level convergence path that blocks unsupported claims, requires evidence for release declarations, and standardizes deployment behavior across web, CLI, and desktop surfaces.

Document Objectives

1. Define the target operating model for all active streams.
2. Establish measurable convergence gates and evidence standards.
3. Align runtime architecture, policy, and QA under one release discipline.
4. Provide implementation-ready work packages with acceptance criteria.
5. Support leadership, technical operations, and stakeholder communication.

Verified Baseline Snapshot

The following baseline was confirmed through direct environment checks during this review cycle:

- Local services active include Python workers, Node surfaces, and Ollama.
- Ollama endpoint is reachable on `127.0.0.1:11434`.
- App surfaces detected listening on local ports including `5000`, `5001`, and `6000`.
- Duplicate listeners were observed on key ports, indicating process duplication risk.
- Existing convergence PDF `v2.0` was verified to be 13 pages, not 50 pages.

Implication: reliability and truthfulness controls must be integrated into routine operations before any production-ready declaration.

Strategic Direction

COMET LEAP advances a local-first, evidence-first stack with explicit boundaries:

- Local inference first; cloud fallback only by explicit policy.

- Human operator control over all mutating actions.
- Deterministic QA gates before release claims.
- Cross-repo convergence with rollback-safe sequencing.
- Strong separation between verified status and aspirational roadmap items.

This direction is designed to maximize operational continuity and minimize failure from tool drift, policy drift, or claim drift.

System Context

The program spans two active repos with different but coupled responsibilities:

- ``human-flourishing-frameworks-scan``: runtime surfaces, policy docs, test suites, and release-facing artifacts.
- ``gm-agent-orchestrator``: orchestration patterns, automation utilities, and PDF generation utilities.

Coupling points include:

- LLM routing policy and operational boundaries.
- Shared release claims and readiness language.
- Build/report pipelines used for stakeholder outputs.

Architecture Model

Runtime Layers

1. Interface Layer

- Web interface at local host ports.
- CLI interaction surfaces.
- Desktop workflow surfaces where applicable.

2. Routing Layer

- Orchestrator-managed provider selection and dispatch.
- Priority order: local providers before remote providers.
- Policy checks before escalation to remote endpoints.

3. Inference Layer

- Local model endpoints (Ollama primary).
- Fallback providers only when policy permits.

4. Persistence Layer

- JSONL event logs as operational record.
- Structured local state directories.
- Optional backup/sync surfaces kept opt-in.

5. Governance Layer

- Guardrail tests.
- Policy manifests.
- Evidence reports linking claims to artifacts.

Control Principles

- Fail closed on ambiguous safety boundaries.
- Prefer deterministic routes over implicit behavior.
- Ensure one authoritative statement of runtime status per release cycle.
- Require timestamped command evidence for all material declarations.

Convergence Streams

Stream A: Core Runtime

Goal: stable local-first interaction surfaces with deterministic startup behavior.

Current state:

- Functional local endpoints and active process footprint.
- Evidence of duplicate processes indicates concurrency hygiene gap.

Actions:

- Add startup preflight to detect existing listeners on required ports.
- Enforce singleton process lock for each core app role.
- Introduce shutdown hooks that clear PID records cleanly.

Acceptance:

- Single active listener per required app port.
- Startup script exits with explicit error when conflict detected.
- Runtime status endpoint returns one canonical process map.

Stream B: Orchestration and Policy

Goal: hard boundaries for provider routing, remote trust, and release claims.

Current state:

- Orchestrator policy exists but includes mixed confidence language and some formatting corruption.
- Convergence policy v2 created with deterministic checklist.

Actions:

- Normalize policy docs to UTF-8 clean text.
- Standardize policy language for allowed, denied, and conditional actions.
- Bind policy checklist to release report generation.

Acceptance:

- All policy documents parse cleanly and contain no encoding artifacts.
- Every production-ready claim references convergence checklist completion.
- Policy and runbook versions are aligned per release tag.

Stream C: QA and Guardrails

Goal: reproducible quality gates that catch policy and behavior regressions early.

Current state:

- Multiple targeted suites pass.

- Foundry hardening suite currently fails due to term/policy mismatch in tracked content.

Actions:

- Resolve policy intent mismatch (legacy term handling vs new enforcement scope).
- Add explicit test fixture boundaries for legacy documentation.
- Require test matrix summary in evidence report.

Acceptance:

- Required suites pass consistently from clean checkout.
- Hardening policy behavior documented and test expectations aligned.
- CI status summary auto-populated into convergence evidence file.

Stream D: Documentation and Reporting

Goal: stakeholder-facing reports that are fully substantiated by repository truth.

Current state:

- Existing v2.0 report had placeholder/filler content.
- Need a real, auditable master plan suitable for printing and review.

Actions:

- Replace placeholder content with full operational narrative.
- Cross-reference all material assertions to file/test/process evidence.
- Establish report quality checklist before PDF generation.

Acceptance:

- Report contains no filler sections.
- All major status claims map to verifiable artifacts.
- PDF output passes readability and pagination review.

QA Framework

Required Test Gates

1. Schema/source/lore integrity suite.
2. CI workflow guardrail suite.
3. Data-center anchor suite.
4. Foundry hardening suite.

Test Discipline

- Run suites in fixed order.
- Capture timestamped outputs.
- Fail fast on first blocker for mutating releases.
- Summarize pass/fail in evidence report with unresolved blockers listed.

Defect Handling

- Classify as blocker, major, or advisory.
- For blockers: no production-ready language allowed.
- For majors: release allowed only with explicit waiver section.
- For advisories: tracked in backlog with owner and due date.

Evidence and Truthfulness Standard

To eliminate hallucinated delivery claims, this plan adopts a strict claim protocol:

- A claim is valid only if backed by one of:
 - Existing file path and content check.
 - Test output with timestamp.
 - Commit SHA and repository state.
 - Runtime process or listener proof.
 - Print spooler job evidence (for physical print claims).

All unsupported claims must be reclassified as roadmap intent.

Data and Storage Design

Operational Records

- JSONL remains preferred event format for append-only operational logs.
- Evidence reports are markdown artifacts with embedded command output blocks.
- PDF reports are derivative artifacts generated from markdown source of truth.

Retention Guidance

- Keep at least last 14 convergence evidence reports.
- Keep release-linked reports indefinitely or archive per policy.
- Avoid mixing generated artifacts with source where possible.

Integrity Controls

- Include creation timestamp and generator command in report metadata.
- Include source file hash in report appendix for critical docs.
- Keep reproducible build command for PDF generation.

Deployment and SRE Operating Model

Startup Contract

- Validate dependencies.
- Validate port availability.
- Validate policy config parse.
- Start services in deterministic order.
- Record startup result events.

Health Contract

- Health endpoint includes version, uptime, provider status, and policy mode.
- Watchdog includes restart reason and count window.
- Duplicate process detection alert threshold configured.

Incident Contract

- Every incident gets: summary, impact, root cause hypothesis, mitigation, and follow-up.
- No silent restarts without log entries.

Security and Privacy Controls

Required Controls

- No implicit external tunnel trust.
- Secrets from environment only; never hardcoded in docs/scripts.
- Explicit user consent for any non-local data flow.
- Minimum privileged token separation for write-level endpoints.

Threat Focus

- Prompt injection through untrusted external text.
- Over-privileged automation scripts.
- Unverified third-party skills/plugins.
- Misleading readiness claims causing unsafe operator actions.

Control Enhancements

- Add provenance tags in critical pipeline docs.
- Add lightweight doc scanner for sensitive patterns and stale placeholders.

- Validate print/report outputs do not leak secrets.

Financial and Capacity Planning

Cost Model

- Baseline objective remains local inference for cost containment.
- Cloud fallback reserved for constrained scenarios.
- Capacity planning tied to local machine constraints and latency targets.

Resource Planning

- CPU/GPU utilization monitoring added to capacity review cadence.
- Model selection matrix by workload type and response-time SLA.
- Incident budget for downtime/recovery windows.

Governance and Roles

Roles

- Operator: runs startup, validates runtime, approves release wording.
- Engineer: implements changes, resolves blockers, maintains tests.
- Reviewer: validates evidence report and claim hygiene.

Review Cadence

- Daily: runtime health + duplicate process check.
- Per change set: targeted test suite + evidence update.
- Per release: full convergence checklist and signed report.

Implementation Roadmap (12 Weeks)

Phase 1 (Weeks 1-2): Truth Stabilization

- Normalize active docs and remove encoding corruption.
- Finalize claim-to-evidence template.
- Add process duplication check to startup scripts.

Phase 2 (Weeks 3-5): Guardrail Alignment

- Resolve hardening test/policy mismatch.
- Add CI summary export into evidence report.
- Introduce release wording gate in report generation.

Phase 3 (Weeks 6-8): Runtime Reliability

- Implement singleton locks and clean shutdown records.
- Add restart-rate alerting.
- Validate local-only routing under stress tests.

Phase 4 (Weeks 9-10): Reporting Excellence

- Produce full stakeholder-ready master plan revisions.
- Add appendix with evidence map and provenance.
- Validate printability and visual readability.

Phase 5 (Weeks 11-12): Release Hardening

- Run end-to-end dry run from clean state.
- Conduct failure injection drills.

- Publish final release candidate package with evidence.

KPI Scorecard

Reliability KPIs

- Single-listener compliance rate.
- Mean restart interval.
- Incident detection-to-recovery time.

Quality KPIs

- Required suite pass rate.
- Open blocker count.
- Claim verification coverage percentage.

Delivery KPIs

- Planned vs completed stream milestones.
- Report generation cycle time.
- Number of manual interventions per week.

Risk Register

R1: Policy/Content Mismatch

Impact: high

Likelihood: high

Mitigation: clarify hardening scope and align tests with policy intent.

R2: Duplicate Process Drift

Impact: high

Likelihood: medium

Mitigation: singleton locks, preflight checks, watchdog reason codes.

R3: Claim Inflation

Impact: high

Likelihood: medium

Mitigation: enforce claim-evidence gate in release artifacts.

R4: Toolchain Inconsistency

Impact: medium

Likelihood: medium

Mitigation: standardized commands and version pinning for report generation.

R5: Secret Exposure Through Docs

Impact: high

Likelihood: low

Mitigation: pre-publish scan for token-like strings and env var misuse.

Acceptance Criteria for This Report Version

1. No placeholder or filler sections.
2. Actionable stream plans with measurable acceptance checks.
3. Explicit risk register with mitigations.
4. Governance, operations, QA, and roadmap fully covered.
5. Printable PDF generated from this markdown source.

Appendix A: Verified Source Inventory

- `README.md`
- `ORCHESTRATOR-POLICY.md`
- `UNIFIED-DEPLOYMENT-MANIFEST.md`
- `LOCAL-LLM-STORAGE-FORMAT.md`
- `CONVERGENCE-EVIDENCE-REPORT.md`

- `ORCHESTRATOR-CONVERGENCE-POLICY-v2.md`

Appendix B: Runtime Snapshot Guidance

Capture at minimum:

- Active process summary for Python/Node/Ollama.
- Local listening ports.
- Branch/remotes/HEAD for both repos.

Appendix C: Release Language Guardrail

Allowed phrasing:

- "Validated in this environment with attached evidence."
- "Passes listed tests except blockers noted in this report."

Disallowed phrasing without full proof:

- "Production ready" (unless all required gates pass).
- "Zero hallucinations" (unless claim protocol score is 100% with audit).

Appendix D: Report Build Instructions

Use markdown source as canonical:

- Input: `COMET-LEAP-MASTER-PLAN-v2.1.md`
- Output: `COMET-LEAP-MASTER-PLAN-v2.1.pdf`
- Tool: `scripts/markdown-to-pdf.py`

Appendix E: Next Revision Priorities

1. Integrate live CI check export.
2. Auto-embed test outputs into appendix.
3. Add process graph visualization for runtime topology.
4. Build release candidate checklist dashboard.

Closing Statement

This v2.1 document is the substantive convergence report baseline. It is designed to be executable by engineering teams, auditable by reviewers, and understandable by stakeholders without relying on placeholders or unverifiable claims.